our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Abster
Freefall

SECURITY REVIEW

Date: 3 December 2025

CONTENTS

1. About Shieldify Security **3**

2. Disclaimer **3**

3. About Abster – Freefall **3**

4. Risk classification **3**

4.1 Impact 3

4.2 Likelihood 3

5. Security Review Summary **4**

5.1 Protocol Summary 4

5.2 Scope 4

6. Findings Summary **4**

7. Findings **5**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Abster - Freefall

Freefall is a multiplier-based betting game where players place bets and receive a randomly determined multiplier that affects their payout. The game operates on a simple yet engaging mechanic:

1. Player places a bet: Players stake a specified amount of tokens (ETH or ERC20) within the pool's min/max bet limits
2. Random multiplier selection: Using VRF-based randomness, a multiplier is selected from a weighted probability distribution
3. Payout calculation: The payout is calculated as betAmount × multiplier / 10000 (basis points)
4. Claim winnings: Once the random number is fulfilled, players can claim their winnings

The multiplier system uses weighted packages, allowing administrators to configure different multipliers with specific probability weights. For example, a multiplier of 20000 (2x) might have a higher weight (more likely) than a multiplier of 100000 (10x), creating a balanced risk-reward structure.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable - and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 3 days with a total of 48 hours dedicated to the audit by two researchers from the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one High, three Medium and three Low severity findings. They're mainly related to broken payout logic and misconfigured game parameters.

The Abster team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	**Abster – Freefall**
Repository	freefall-contract
Type of Project	GameFi
Security Review Timeline	3 days
Review Commit Hash	35509deea2637a80b1059ee3ed4c56893818da7d
Fixes Review Commit Hash	b91908956fefd279a6b0f6b7cf38d476e5ff8975

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
src/Freefall.sol	406
Total	**406**

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **High** issues: **1**
- **Medium** issues: **3**
- **Low** issues: **3**
- **Info** issues: **1**

ID	Title	Severity	Status
[H-01]	Claim Allows Payouts From an Arbitrary Token Pool, Not the Game's Token	High	Fixed
[M-01]	Multiplier Configuration Can Break Distribution Invariant and Permanently Freeze Game Resolution and Claims	Medium	Fixed
[M-02]	The `removePool()` Function Permanently Locks User Balances for that Token	Medium	Fixed
[M-03]	In Case of No Referrer, Platform Charges Referral Fee to Themselves Instead of Adding it Back to `remainingAmount`	Medium	Acknowledged
[L-01]	The `addMultiplierPackage()` Is Unusable After Normal Initialization	Low	Fixed
[L-02]	No Upper Bounds on `platformFee` and `referralFee`	Low	Fixed
[L-03]	The `payFees()` Function Returns Invalid Fee When the Referrer or Fee-Recipients Are Not Present	Low	Fixed
[I-01]	`GameResultNotFulfilled` Error Used for Opposite Conditions	Info	Fixed

7. Findings

[H-01] Claim Allows Payouts From an Arbitrary Token Pool, Not the Game's Token

Severity

High Risk

Description

The `claim()` trusts the caller-supplied `_tokenAddress` and never checks if it matches the token used for the game. As a result, a user who played a game in **TokenA** can claim their payout using **TokenB**, draining the liquidity pool of **B** even though they never bet that token.

This is a direct cross-asset theft vector.

Example:

- There are two supported tokens:
  - `tokenA` (e.g. USDC) with a small pool.
  - `tokenB` (e.g. WETH) with a large pool.
- User plays a game using `tokenA`:
  - `createGame(tokenA, ...)` stores `game.tokenAddress = tokenA`.
  - VRNG resolves, sets `game.determinedMultiplier`.
- Payout is determined, say `payout = 1_000e6` (USDC).

Location of the code

File: src/Freefall.sol#L219

```
function claim(address _tokenAddress, string calldata _gameId)
{
    Pool storage pool = liquidityPool[_tokenAddress];        // <--
       attacker controls
    uint256 onChainGameId = offChainGameIdToOnChainGameId[_gameId];
    Game storage game = games[onChainGameId];

    // payout computed from game.'tokenAddresss bet
    uint256 payout = (game.betAmount * game.determinedMultiplier) /
       BASIS_POINTS;

    pool.balance -= payout;                                  // <--
       drains wrong pool
    tokenTransfer(_tokenAddress, payout, msg.sender);        // <-- sends
        attacker chosen token
}
```

Impact

A malicious player can:

- Play a game in a small token (e.g., USDC with a low pool).
- Claim using a token with a large pool (e.g., WETH).
- Drain WETH liquidity entirely.

Recommendation

Bind the claim payout to the token used for the game:

```
require(_tokenAddress == game.tokenAddress, "Invalid token for claim");
```

Then always use `game.tokenAddress` for pool and transfers.

Team Response

Fixed.

[M-01] Multiplier Configuration Can Break Distribution Invariant and Permanently Freeze Game Resolution and Claims

Severity

Medium Risk

Description

The multiplier system relies on a strict cumulative-weight distribution that must fully cover the range `[0, WEIGHT_DENOMINATOR)` for randomness selection to work.

This invariant is explicitly enforced in `initializeDefaultPackages()` through:

```
require(_weights[_weights.length - 1] == WEIGHT_DENOMINATOR);
require(_weights[i] > _weights[i - 1]);
```

However, all other admin update functions ignore this invariant:

- `addMultiplierPackage()` only checks:

  ```
  require(_weight <= WEIGHT_DENOMINATOR);
  require(_weight > previousWeight);
  ```

  Does not enforce last weight == 10_000.

- `updateMultiplierPackage()` only enforces local ordering between neighbours:

  ```
  require(_weight > previousWeight);
  require(_weight < nextWeight);
  ```

  Again, no check for full coverage.

- `removeLastMultiplierPackage()` simply pops the last package:

  ```
  multiplierPackages.pop();
  ```

  This usually removes the only package whose weight == 10000.

Because of these missing checks, admins can unintentionally create a multiplier distribution where the final weight is less than 10_000 or where some intervals are inactive (due to `active=false`).

This produces gaps in the `[0, WEIGHT_DENOMINATOR)` domain.

When VRNG produces a `randomValue` that falls into one of these gaps:

- `selectMultiplierFromRandom()` finds no matching package
- It reverts
- `_onRandomNumberFulfilled()` reverts
- `game.resultFulfilled` is never set
- The request ID is never cleared
- `claim()` will always revert for that game

This permanently locks the user's funds for affected games.

This is not merely an "admin misconfig risk" – the contract enforces the invariant at initialization but fails to enforce it for all later updates, leading to internal inconsistency and game-breaking behavior.

Location of the code

File: src/Freefall.sol#L453

```
function initializeDefaultPackages(uint256[] calldata _multipliers,
    uint256[] calldata _weights) external onlyRole(ADMIN_ROLE) {
    require(!multiplierPackagesInitialized, PackagesAlreadyInitialized())
        ;
    require(_multipliers.length > 0, InvalidMultiplierPackage());
    require(_multipliers.length == _weights.length,
        InvalidWeightConfiguration());
    require(_weights[_weights.length - 1] == WEIGHT_DENOMINATOR,
        InvalidWeightConfiguration());

    for (uint256 i = 0; i < _weights.length; i++) {
        if (i > 0) {
            require(_weights[i] > _weights[i - 1],
                InvalidWeightConfiguration());
        }
    }

    for (uint256 i = 0; i < _multipliers.length; i++) {
        multiplierPackages.push(MultiplierPackage({
            multiplier: _multipliers[i],
            weight: _weights[i],
            active: true
        }));
    }

    multiplierPackagesInitialized = true;

    emit MultiplierPackagesInitialized(multiplierPackages.length);
}
```

But not enforced here:

- `addMultiplierPackage()`
- `updateMultiplierPackage()`
- `removeLastMultiplierPackage()`

Selector that fails when the invariant is broken:

```
if (active && randomValue < weight) return multiplier;

revert NoPackagesConfigured();
```

Randomness fulfilment that becomes permanently broken:

```
uint256 selectedMultiplier = selectMultiplierFromRandom(randomNumber); //
    revert
// game.resultFulfilled is never set
```

Impact

If the admin:

- removes the last package,
- updates a package weight incorrectly,
- disables the catch-all package (`active = false`),
- or creates any weight sequence where the last weight < 10_000... ...then some percentage of all games will become permanently unresolvable.

Consequences:

- `_onRandomNumberFulfilled()` repeatedly reverts for those values.
- `game.resultFulfilled` stays `false`.
- `claim()` becomes permanently impossible (`GameResultNotFulfilled`).
- User funds for those games are stuck forever.
- Depending on the configuration, this can impact:
  - A subset of random values,
  - A large portion of users,
  - Or all new games until the distribution is fixed.

Recommendation

Enforce the full cumulative distribution invariant after every update, not only during initialization:

- The last multiplier package must always have:

```
weight == WEIGHT_DENOMINATOR;
active == true;
```

- All weights must remain strictly increasing.
- If packages are added/removed/updated, validate the entire array before accepting changes.

Team Response

Fixed.

[M-02] The `removePool()` Function Permanently Locks User Balances for that Token

Severity

Medium Risk

Description

The `removePool()` deletes the pool entry but does not clear or migrate `userBalances`.

After removal, `isTokenSupported()` returns false for that token, which blocks:

- `withdraw()`

- `deposit()`
- `claim()`
- `addLiquidity()` / `removeLiquidity()`
- `createGame()`

Users with positive balances for that token can never withdraw them again.

There is no escape path for users or admins.

Location of the code

File: src/Freefall.sol#L444

```
function removePool(address _tokenAddress) public onlyRole(ADMIN_ROLE) {
    require(liquidityPool[_tokenAddress].balance == 0, PoolHasBalance());
    delete liquidityPool[_tokenAddress];
    emit PoolRemoved(_tokenAddress);
}
```

File: src/Freefall.sol#L63

```
modifier isTokenSupported(address _tokenAddress) {
    require(liquidityPool[_tokenAddress].isActive, PoolNotActive());
    _;
}
```

Impact

If an admin removes a pool, even accidentally, all users holding that token are permanently stuck. Their funds remain inside the contract with no method to withdraw.

Recommendation

Before deleting a pool, ensure that no `userBalances` exist for that token or implement a recovery function that allows withdrawals even when the pool is inactive.

Team Response

Fixed.

[M-03] In Case of No Referrer, Platform Charges Referral Fee to Themselves Instead of Adding it Back to `remainingAmount`

Severity

Medium Risk

Description

The total fee is calculated as a fraction of the bet amount. It calculates fees as such:

```
uint256 platformFeeAmount = _amount * platformFee / 10000;
uint256 referralFeeAmount = _amount * referralFee / 10000;
```

If a referrer is present, the referrer fee amount is sent to that address or otherwise, the `referralFeeAmount` is added to the `platformFeeAmount`.

As a result, the referralFeeAmount is charged to the user, regardless of whether a referrer is present or not.

Location of Affected Code

File: src/Freefall.sol#L310

```
function payFees(uint256 _amount, address _tokenAddress, address
   _referrer)
    internal
    returns (uint256 remainingAmount, uint256 totalFees)
{
    // code
    uint256 platformFeeAmount = _amount * platformFee / 10000;
    uint256 referralFeeAmount = _amount * referralFee / 10000;

    totalFees = platformFeeAmount + referralFeeAmount;

    remainingAmount = _amount - totalFees;

    if (_referrer != address(0)) {
        tokenTransfer(_tokenAddress, referralFeeAmount, _referrer);
    } else {
        platformFeeAmount += referralFeeAmount; //@audit-issue platform
           charges referral fee to themselves, in case of no referrer
    }

    // code
}
```

Impact

The `referralFeeAmount` is charged to the user, regardless of whether a referrer is present or not.

Recommendation

Consider applying the following change:

```
if (_referrer != address(0)) {
    tokenTransfer(_tokenAddress, referralFeeAmount, _referrer);
} else {
-   platformFeeAmount += referralFeeAmount;
+   remainingAmount += referralFeeAmount;
}
```

Team Response

Acknowledged.

[L-01] The `addMultiplierPackage()` Is Unusable After Normal Initialization

Severity

Low Risk

Description

After a normal call to `initializeDefaultPackages()`, the last multiplier package is required to have `weight == WEIGHT_DENOMINATOR` (10_000).

Later, `addMultiplierPackage()` requires any new package to have `_weight > lastWeight` and `_weight <= WEIGHT_DENOMINATOR`.

In a sane config (last weight = 10_000), these conditions are mutually exclusive:

- `_weight > 10000` AND `_weight <= 10000` impossible.

This makes `addMultiplierPackage()` effectively unusable after initialization and turns it into dead code.

Location of the code

File: src/Freefall.sol#L453

```
function initializeDefaultPackages(uint256[] calldata _multipliers,
    uint256[] calldata _weights) external onlyRole(ADMIN_ROLE) {
    require(!multiplierPackagesInitialized, PackagesAlreadyInitialized())
        ;
    require(_multipliers.length > 0, InvalidMultiplierPackage());
    require(_multipliers.length == _weights.length,
        InvalidWeightConfiguration());
    require(_weights[_weights.length - 1] == WEIGHT_DENOMINATOR,
        InvalidWeightConfiguration()); // <-----

    for (uint256 i = 0; i < _weights.length; i++) {
        if (i > 0) {
            require(_weights[i] > _weights[i - 1],
                InvalidWeightConfiguration());
        }
    }

    for (uint256 i = 0; i < _multipliers.length; i++) {
        multiplierPackages.push(MultiplierPackage({
            multiplier: _multipliers[i],
            weight: _weights[i],
            active: true
        }));
    }

    multiplierPackagesInitialized = true;

    emit MultiplierPackagesInitialized(multiplierPackages.length);
}
```

File: src/Freefall.sol#L482

```
function addMultiplierPackage(uint256 _multiplier, uint256 _weight)
    external onlyRole(ADMIN_ROLE) {
    require(_weight <= WEIGHT_DENOMINATOR, InvalidWeightConfiguration());
        // <-----

    if (multiplierPackages.length > 0) {
        require(_weight > multiplierPackages[multiplierPackages.length -
            1].weight, InvalidWeightConfiguration()); // <-----
    }

    multiplierPackages.push(MultiplierPackage({
        multiplier: _multiplier,
        weight: _weight,
        active: true
    }));

    emit MultiplierPackageAdded(multiplierPackages.length - 1,
        _multiplier, _weight);
}
```

Recommendation

Restrict `addMultiplierPackage()` to be used only before initialization (e.g. `require(!multiplierPackagesInitialized)`) or redesign the function so it can safely append packages after a `weight == WEIGHT_DENOMINATOR` tail (for example, by renormalising or changing the weight semantics).

Team Response

Fixed.

[L-02] No Upper Bounds on `platformFee` and `referralFee`

Severity

Low Risk

Description

The `platformFee` and `referralFee` are meant to be basis-point style parameters (denominator 10_000), but there is no upper bound enforced when updating them. This allows setting values greater than 10_000 bp (100%) or unreasonably large values, which:

- Makes configuration error-prone.
- Can easily lead to a situation where total fee >= 100% of the bet or to very large fees that make the game unusable.

Location of the code

File: src/Freefall.sol#L373

```
uint256 public platformFee;
uint256 public referralFee;

function updatePlatformFee(uint256 _platformFee) public onlyRole(
   ADMIN_ROLE) {
    platformFee = _platformFee;
}

function updateReferralFee(uint256 _referralFee) public onlyRole(
   ADMIN_ROLE) {
    referralFee = _referralFee;
}
```

Recommendation

Enforce upper bounds consistent with the basis-point model, for example:

```
function updatePlatformFee(uint256 _platformFee) public onlyRole(
    ADMIN_ROLE) {
     require(_platformFee <= 10_000, "platformFee too high");
     platformFee = _platformFee;
}

function updateReferralFee(uint256 _referralFee) public onlyRole(
    ADMIN_ROLE) {
     require(_referralFee <= 10_000, "referralFee too high");
     referralFee = _referralFee;
}
```

Optionally and a little bit more complex mitigation is to enforce `platformFee + referralFee <= 10_000` if the intent is "fees never exceed 100% of the bet."

Enforce upper bounds in the constructor too.

Team Response

Fixed.

[L-03] The `payFees()` Function Returns Invalid Fee, When the Referrer or Fee-Recipients Are Not Present

Severity

Low Risk

Description:

In `payFees()`, `totalFees` is initially computed as `platformFeeAmount + referralFeeAmount` and `remainingAmount` as `_amount - totalFees`. However, depending on the presence of a referrer and fee recipients, part or all of these fees may be returned to `remainingAmount` without updating `totalFees`.

Concretely:

- If a referrer exists but `feeRecipients.length == 0`, only the referral fee is actually transferred, but `totalFees` still equals `platformFee + referralFee`.
- If no referrer exists and `feeRecipients.length == 0`, no fees are transferred at all, but `totalFees` still equals `platformFee + referralFee`. Here, totalFees != _amount - remainingAmount' and does not match the sum of transferred fees.

Location of Affected Code

File: src/Freefall.sol#L310

```
function payFees(uint256 _amount, address _tokenAddress, address
   _referrer)
    internal
    returns (uint256 remainingAmount, uint256 totalFees)
{
    uint256 platformFeeAmount = _amount * platformFee / 10000;
    uint256 referralFeeAmount = _amount * referralFee / 10000;
    //@note totalFees contains both platform and referral fees
    totalFees = platformFeeAmount + referralFeeAmount;
    //@note remaining amount = amount - fee (platform + referral)
    remainingAmount = _amount - totalFees;

    // code

    //@audit-issue returns total fee (platform + referral) even if no fee
        recipients or referral fee is charged.
    return (remainingAmount, totalFees);
}
```

Impact

Incorrect accounting of fees

Recommendation:

Compute `totalFees` only from the amounts actually transferred out (referrer + recipients), and set `remainingAmount = _amount - totalFees` once at the end. Do not pre-subtract fees that may later be refunded.

```
function payFees(uint256 _amount, address _tokenAddress, address
   _referrer)
    internal
    returns (uint256 remainingAmount, uint256 totalFees)
{
    uint256 platformFeeAmount = (_amount * platformFee) / 10000;
    uint256 referralFeeAmount = (_amount * referralFee) / 10000;

    // 1) Referral fee
    if (_referrer != address(0) && referralFeeAmount > 0) {
        totalFees += referralFeeAmount;
        tokenTransfer(_tokenAddress, referralFeeAmount, _referrer);
    } else {
        // No referrer → referral fee gets rolled into platform portion
        platformFeeAmount += referralFeeAmount;
    }

    // 2) Platform fee
    uint256 totalFunders = feeRecipients.length;

    if (totalFunders > 0 && platformFeeAmount > 0) {
        totalFees += platformFeeAmount;
        uint256 funderFeeAmount = platformFeeAmount / totalFunders;
        uint256 dust = platformFeeAmount - funderFeeAmount * totalFunders
            ;
        for (uint256 i = 0; i < totalFunders; i++) {
            if (i == totalFunders - 1) {
                tokenTransfer(_tokenAddress, funderFeeAmount + dust,
                    feeRecipients[i]);
            } else {
                tokenTransfer(_tokenAddress, funderFeeAmount,
                    feeRecipients[i]);
            }
        }
    } else {
        // No recipients → no platform fee taken
        // nothing to do, platformFeeAmount stays in remainingAmount
    }

    remainingAmount = _amount - totalFees;
}
```

Team Response

Fixed.

[I-01] `GameResultNotFulfilled` Error Used for Opposite Conditions

Severity

Informational Risk

Description

The same custom error, `GameResultNotFulfilled`, is used for two opposite situations:

- When a user calls `claim()` before the result is ready (`resultFulfilled == false`).
- When VRNG (or any caller) tries to fulfil a result that is already finalized (`resultFulfilled == true`).

Both revert with `GameResultNotFulfilled()`, even though the root causes are different ("not fulfilled yet" vs "already fulfilled").

This makes logs and debugging ambiguous and complicates off-chain monitoring.

Location of the code

File: src/Freefall.sol#L229

```
// claim()
require(game.resultFulfilled, GameResultNotFulfilled());

// _onRandomNumberFulfilled()
require(!game.resultFulfilled, GameResultNotFulfilled());
```

Recommendation

Split into two explicit errors, for example:

```
error GameResultNotFulfilled();
error GameResultAlreadyFulfilled();
```

Use `GameResultNotFulfilled` in `claim()`, and `GameResultAlreadyFulfilled` in `_onRandomNumberFulfilled()`.

Team Response

Fixed.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!